

RESOURCES & TECHNICAL SETUP



What You Have Access To

Free GPU Resources

1. Google Colab (Recommended for Beginners)

- **GPU:** Tesla T4 (16GB)
- **Time Limit:** ~12 hours per session
- **Cost:** Free!
- **Best for:** Tier 1 projects
- **Link:** colab.research.google.com

Pros:

- No setup required
- Pre-installed libraries
- Easy to share
- Integrates with Google Drive

Cons:

- Sessions timeout
- Can't run overnight
- Need to save work frequently

Setup:

```
# Test GPU
!nvidia-smi

# Install packages
!pip install ultralytics torch torchvision opencv-python
```

2. Kaggle Notebooks

- **GPU:** P100 or T4
- **Time Limit:** 30 hours per week
- **Cost:** Free!
- **Best for:** Tier 1 & 2 projects

- **Link:** kaggle.com/code

Pros:

- More GPU time than Colab
- Access to Kaggle datasets
- Can run longer jobs

Cons:

- Weekly time limit
- Slightly less flexible than Colab

Setup:

```
# Enable GPU in Settings → Accelerator → GPU
# Import common libraries (pre-installed)
import torch
print(torch.cuda.is_available()) # Should be True
```

3. Heidi Platform (Adaptive Computing)

- **GPU:** A100 (40GB)
- **Time Limit:** Ask instructor for allocation
- **Cost:** Free for students (limited hours)
- **Best for:** Tier 2 & 3 projects
- **Access:** Request from instructor

Pros:

- Powerful GPU
- Longer sessions
- More memory

Cons:

- Limited allocation
- Requires instructor approval
- Need to learn cluster computing

Free API Services

1. Hugging Face Inference API

- **What:** Run models without your own GPU
- **Cost:** Free tier available

- **Best for:** VLM, text-to-image, embeddings
- **Link:** huggingface.co/inference-api

Example:

```
import requests

API_URL = "https://api-inference.huggingface.co/models/Salesforce/blip-image-captioning"
headers = {"Authorization": "Bearer YOUR_HF_TOKEN"}

def query(filename):
    with open(filename, "rb") as f:
        data = f.read()
    response = requests.post(API_URL, headers=headers, data=data)
    return response.json()

output = query("image.jpg")
print(output)
```

2. Replicate

- **What:** Run AI models via API
- **Cost:** Free credits, then pay-per-use
- **Best for:** Quick testing of models
- **Link:** replicate.com

Example:

```
import replicate

output = replicate.run(
    "stability-ai/stable-diffusion:latest",
    input={"prompt": "a photo of a cat"}
)
print(output)
```

3. OpenAI API (Optional, Paid)

- **What:** GPT-4V for vision tasks
- **Cost:** ~\$5-10 for typical project
- **Best for:** Tier 2/3 VLM projects
- **Link:** platform.openai.com

Only use if: You're doing a VLM project and want state-of-the-art results

Pre-trained Model Sources

1. Ultralytics (YOLO)

- **Free:** Yes
- **Models:** YOLOv8, YOLOv5, SAM
- **Link:** github.com/ultralytics/ultralytics

```
from ultralytics import YOLO

# All free, pre-trained models:
model = YOLO('yolov8n.pt') # Nano (fastest)
model = YOLO('yolov8s.pt') # Small
model = YOLO('yolov8m.pt') # Medium
model = YOLO('yolov8l.pt') # Large
model = YOLO('yolov8x.pt') # Extra Large (most accurate)
```

2. Hugging Face Models

- **Free:** Yes
- **Models:** 1000s of CV models
- **Link:** huggingface.co/models

Popular models:

```
from transformers import pipeline

# Image classification
classifier = pipeline("image-classification",
                      model="microsoft/resnet-50")

# Object detection
detector = pipeline("object-detection",
                    model="facebook/detr-resnet-50")

# Image captioning
captioner = pipeline("image-to-text",
                     model="Salesforce/blip-image-captioning-base")
```

3. TorchVision Models

- **Free:** Yes
- **Models:** Classic CV architectures
- **Link:** pytorch.org/vision

```
import torchvision.models as models

# Pre-trained on ImageNet
resnet = models.resnet50(pretrained=True)
efficientnet = models.efficientnet_b0(pretrained=True)
vgg = models.vgg16(pretrained=True)
mobilenet = models.mobilenet_v2(pretrained=True)
```

Learning Resources

Official Documentation

1. PyTorch

- Link: pytorch.org/tutorials
- Best for: Understanding deep learning basics
- Start here: "Introduction to PyTorch"

2. Ultralytics YOLO

- Link: docs.ultralytics.com
- Best for: Object detection
- Very beginner-friendly!

3. Hugging Face

- Link: huggingface.co/docs
- Best for: Transformers, VLMs
- Excellent examples

Video Tutorials

1. Object Detection

- YouTube: "YOLOv8 Tutorial" by Nicolai Nielsen
- YouTube: "Object Detection from Scratch" by Aladdin Persson

2. Image Classification

- YouTube: "PyTorch Image Classification" by Python Engineer
- Coursera: "Convolutional Neural Networks" by Andrew Ng (free audit)

3. Computer Vision Fundamentals

- YouTube: "First Principles of Computer Vision" by Shree Nayar
 - Stanford CS231n (free on YouTube)
-

Code Examples & Repos

1. YOLOv8 Examples

- github.com/ultralytics/ultralytics/tree/main/examples
- Copy-paste friendly examples for common tasks

2. PyTorch Image Models (timm)

- github.com/huggingface/pytorch-image-models
- Hundreds of model implementations

3. Awesome Computer Vision

- github.com/jbhuang0604/awesome-computer-vision
 - Curated list of CV resources
-

Datasets

For Object Detection:

COCO (Common Objects in Context)

- **Size:** 330K images, 80 classes
- **Link:** cocodataset.org
- **Best for:** General object detection

Roboflow Universe

- **Size:** 100,000+ datasets
- **Link:** universe.roboflow.com
- **Best for:** Specific domains (retail, medical, industrial)

Open Images

- **Size:** 9M images, 600 classes
 - **Link:** storage.googleapis.com/openimages/web/index.html
 - **Best for:** Large-scale experiments
-

For Classification:

ImageNet

- **Size:** 14M images, 1000 classes

- **Link:** image-net.org
- **Best for:** Pre-training models

Food-101

- **Size:** 101K images, 101 food classes
- **Link:** kaggle.com/datasets/dansbecker/food-101
- **Best for:** Food classification

iNaturalist

- **Size:** 5M images, 10K species
 - **Link:** inaturalist.org
 - **Best for:** Nature/wildlife
-

For Segmentation:

Cityscapes

- **Size:** 25K images, 30 classes
- **Link:** cityscapes-dataset.com
- **Best for:** Autonomous driving, urban scenes

ADE20K

- **Size:** 25K images, 150 classes
 - **Link:** groups.csail.mit.edu/vision/datasets/ADE20K
 - **Best for:** Scene parsing
-

For VLM Projects:

Visual Question Answering (VQA)

- **Size:** 265K images, 1M+ questions
- **Link:** visualqa.org
- **Best for:** VQA systems

COCO Captions

- **Size:** 330K images, 5 captions each
 - **Link:** cocodataset.org/#captions-2015
 - **Best for:** Image captioning
-

Required Software

Python Environment

Option 1: Anaconda (Recommended)

```
# Download from: anaconda.com/download

# Create environment
conda create -n cv_project python=3.10
conda activate cv_project

# Install packages
conda install pytorch torchvision torchaudio -c pytorch
pip install ultralytics opencv-python matplotlib jupyter
```

Option 2: pip + venv

```
# Create virtual environment
python -m venv cv_env

# Activate (Windows)
cv_env\Scripts\activate

# Activate (Mac/Linux)
source cv_env/bin/activate

# Install packages
pip install torch torchvision opencv-python ultralytics jupyter matplotlib
```

Core Libraries

Create `requirements.txt`:

```
# Deep Learning
torch>=2.0.0
torchvision>=0.15.0
ultralytics>=8.0.0

# Computer Vision
opencv-python>=4.8.0
Pillow>=10.0.0

# Data & Visualization
numpy>=1.24.0
matplotlib>=3.7.0
```



```
pandas>=2.0.0

# Optional: Transformers (for VLM)
transformers>=4.30.0

# Optional: Additional tools
scikit-learn>=1.3.0
tqdm>=4.65.0
```

Install all:

```
pip install -r requirements.txt
```

Development Tools

1. VS Code (Recommended)

- Free, lightweight IDE
- Download: code.visualstudio.com
- Extensions to install:
 - Python
 - Jupyter
 - GitHub Copilot (optional)

2. Jupyter Lab

```
pip install jupyterlab
jupyter lab
```

3. Git

- Download: git-scm.com
- Essential for version control



Hardware Requirements

Minimum (For Development)

- **CPU:** Any modern processor
- **RAM:** 8GB
- **Storage:** 10GB free space
- **GPU:** Not required (use Colab!)

Recommended (For Local Training)

- **CPU:** Multi-core (4+ cores)
- **RAM:** 16GB
- **Storage:** 50GB SSD
- **GPU:** NVIDIA GPU with 6GB+ VRAM

Note: Most students will use Colab/Kaggle and don't need local GPU!

Code Templates

YOLOv8 Object Detection

Complete working example:

```
# 1. Install
!pip install ultralytics

# 2. Import
from ultralytics import YOLO

# 3. Load model
model = YOLO('yolov8n.pt')

# 4. Train on custom data
results = model.train(
    data='data.yaml',
    epochs=50,
    imgsz=640,
    batch=16
)

# 5. Validate
metrics = model.val()
print(f"mAP50: {metrics.box.map50:.3f}")

# 6. Predict
results = model('image.jpg')
results[0].show()

# 7. Export (optional)
model.export(format='onnx')
```

PyTorch Image Classification

Transfer learning template:

```
import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# 1. Load pre-trained model
model = models.resnet50(pretrained=True)

# 2. Modify final layer for your classes
num_classes = 10 # Change to your number
model.fc = nn.Linear(model.fc.in_features, num_classes)

# 3. Data preprocessing
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])

# 4. Load data
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# 5. Training loop
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    for images, labels in train_loader:
        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# 6. Save model
torch.save(model.state_dict(), 'model.pth')
```

Hugging Face VLM

Image captioning example:

```
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image

# 1. Load model
processor = BlipProcessor.from_pretrained(
    "Salesforce/blip-image-captioning-base"
)
model = BlipForConditionalGeneration.from_pretrained(
    "Salesforce/blip-image-captioning-base"
)

# 2. Load image
image = Image.open("image.jpg")

# 3. Process and generate
inputs = processor(image, return_tensors="pt")
outputs = model.generate(**inputs)
caption = processor.decode(outputs[0], skip_special_tokens=True)

print(f"Caption: {caption}")
```

? Troubleshooting Common Setup Issues

"pip install fails"

```
# Try upgrading pip
python -m pip install --upgrade pip

# Try with --user flag
pip install --user package-name
```

```
# Clear cache  
pip cache purge
```

"CUDA not available"

```
import torch  
print(torch.cuda.is_available()) # Returns False  
  
# Solutions:  
# 1. Use CPU (slower but works)  
device = 'cpu'  
  
# 2. Use Colab/Kaggle (free GPU)  
  
# 3. Reinstall PyTorch with CUDA:  
# Visit: pytorch.org/get-started/locally/
```

"Out of memory"

```
# Reduce batch size  
batch_size = 8 # or 4, or 2  
  
# Use gradient accumulation  
# Instead of batch=32, use batch=8 with 4 accumulation steps  
  
# Clear cache  
torch.cuda.empty_cache()
```

"ModuleNotFoundError"

```
# Make sure you're in the right environment  
conda activate cv_project  
  
# Install missing package  
pip install package-name  
  
# Check installed packages  
pip list
```



Quick Reference

Essential Commands

```
# Activate environment
conda activate cv_project

# Install new package
pip install package-name

# Check GPU
nvidia-smi

# Start Jupyter
jupyter notebook

# Git commands
git clone [url]
git add .
git commit -m "message"
git push
```

Important Links

- **PyTorch:** pytorch.org
- **Ultralytics:** ultralytics.com
- **Hugging Face:** huggingface.co
- **Colab:** colab.research.google.com
- **Kaggle:** kaggle.com
- **Roboflow:** roboflow.com
- **GitHub:** github.com

Need more help? Ask instructor or check documentation! 🚀

Start coding or [generate](#) with AI.

